

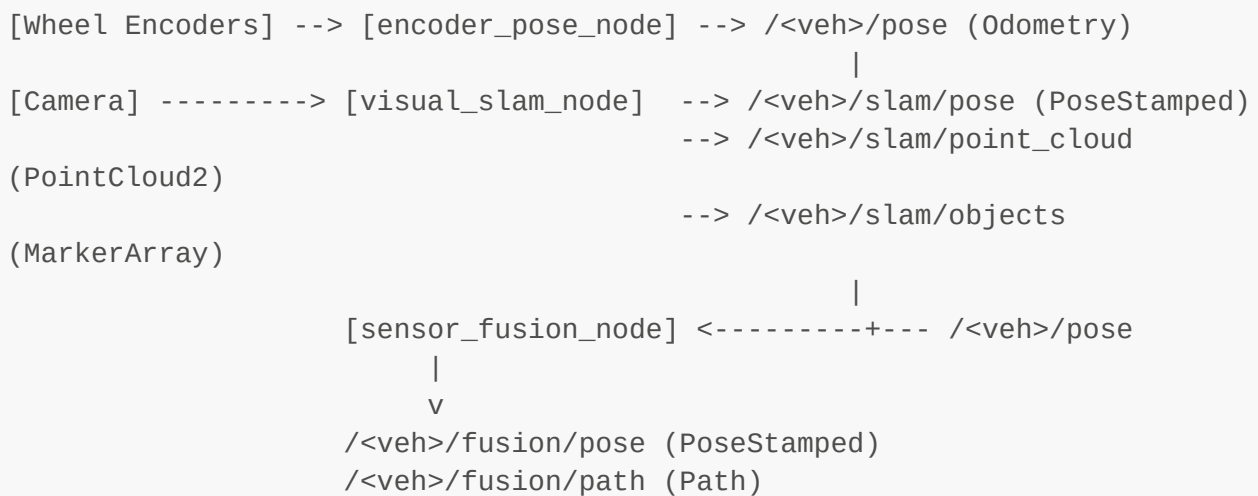
Technisch Verslag – Odometry en SLAM

Jesse Coenraad

Datum: mei 2026

1. Systeemarchitectuur

Het systeem bestaat uit drie ROS-nodes die via topics met elkaar communiceren. Elke node heeft één verantwoordelijkheid en ze zijn zo ontworpen dat je ze ook los van elkaar kunt draaien voor testdoeleinden.



Er zijn drie launchers beschikbaar: **odometry.sh**, **slam.sh** en **default.sh**. De default launcher start alle drie de nodes tegelijk via een gecombineerd launch-bestand.

2. Implementatie per taak

Taak 1 – Odometry

De odometry node leest encoder ticks van het linker- en rechterwiel en berekent hieruit de positie en oriëntatie van de robot. Dit werkt in twee stappen.

Eerst wordt per wiel de hoekverandering berekend:

$$\Delta\phi = (\text{ticks_huidig} - \text{ticks_vorig}) / \text{resolutie} \times 2\pi$$

De resolutie is het aantal ticks per volledige rotatie (135 voor de DB21). Daarna wordt de pose bijgewerkt via het differentieel aandrijvingsmodel:

$$\begin{aligned} d_{\text{links}} &= \Delta\phi_{\text{links}} \times R \\ d_{\text{rechts}} &= \Delta\phi_{\text{rechts}} \times R \end{aligned}$$

```

d      = (d_links + d_rechts) / 2
Δθ     = (d_rechts - d_links) / baseline

x_nieuw = x + d × cos(θ + Δθ/2)
y_nieuw = y + d × sin(θ + Δθ/2)
θ_nieuw = θ + Δθ

```

De pose wordt gepubliceerd als `nav_msgs/Odometry` op `/<veh>/pose`. Om de gereden route te kunnen visualiseren in RViz wordt ook een `nav_msgs/Path` bijgehouden die elke nieuwe pose toevoegt.

Taak 2 – Visual SLAM

De SLAM-node verwerkt elk cameraframe in drie stappen: feature detectie, bewegingsschatting en kaartopbouw.

Feature detectie en matching

Per frame worden ORB-features gedetecteerd (`cv2.ORB_create`, standaard 5000 features). ORB combineert detectie en beschrijving in één stap en is bestand tegen rotatie. Tussen twee opeenvolgende frames worden features gematcht met een Brute Force Matcher op Hamming-afstand. Slechte matches worden gefilterd met de Fundamental Matrix via RANSAC.

Bewegingsschatting

Uit de Fundamental Matrix en de cameramatrix K wordt de Essential Matrix berekend:

$$E = K^T \times F \times K$$

`cv2.recoverPose` geeft de rotatiematrix R en translatievector t terug. Dit is de beweging van het vorige naar het huidige frame, op schaal tot één (monoculaire ambiguïteit).

Kaartopbouw

De gefilterde feature-correspondentie wordt getrianguleerd met `cv2.triangulatePoints`. Punten die achter een van de camera's vallen (negatieve diepte) worden verwijderd. De overblijvende 3D-punten worden naar het wereldframe getransformeerd via de geaccumuleerde camera-pose en gepubliceerd als `PointCloud2`.

Objectdetectie

Naast de feature-gebaseerde kaart detecteert de node objecten via HSV-kleurdrempelwaarden. Duckies (geel), rode stoplichten, groene stoplichten en andere duckiebots (blauw) worden als `MarkerArray` gepubliceerd. Via de debug-image topic (`slam/debug/image/compressed`) zijn de gedetecteerde ORB-keypoints en objecten direct zichtbaar in een image viewer.

Taak 3 – Sensorfusie (EKF)

De sensorfusie-node combineert odometry en SLAM via een Extended Kalman Filter met toestandsvector $[x, y, \theta]$.

Predictie (odometry)

Bij elke nieuwe odometrymeting wordt het delta berekend ten opzichte van de vorige meting. Met het

differentieel aandrijvingsmodel wordt de toestand voorspeld en de covariantiematrix bijgewerkt:

$$P_{\text{pred}} = F \times P \times F^T + Q$$

waarbij F de Jacobiaan van het bewegingsmodel is en Q de procesruis (wielslip, encoderfouten).

Correctie (SLAM heading)

De SLAM-node publiceert de geaccumuleerde camera-pose als **PoseStamped**. Hieruit wordt de yaw-hoek geëxtraheerd als observatie. De innovatie (verschil tussen gemeten en voorspelde hoek) wordt gebruikt voor de EKF-update:

$$\begin{aligned} y &= \theta_{\text{slam}} - \theta_{\text{pred}} \\ K &= P \times H^T \times (H \times P \times H^T + R)^{-1} \\ x &= x_{\text{pred}} + K \times y \end{aligned}$$

Om instabiele correcties te voorkomen wordt **innovation gating** toegepast: als de afwijking groter is dan 30° , wordt de SLAM-correctie overgeslagen. Dit is nodig omdat de SLAM-yaw in cameracoördinaten zit en niet direct overeenkomt met de robotoriëntatie zonder extrinsieke kalibratie.

3. Ontwerpkeuzes en afwegingen

ORB in plaats van Shi-Tomasi + Lucas-Kanade

De eerste implementatie gebruikte Shi-Tomasi corners met optische stroom. Dit werkte instabiel bij snelle bewegingen omdat punten snel verloren gingen tussen frames. ORB met BFMatcher vergelijkt features expliciet op beschrijvers, wat robuuster is en bovendien echte 3D-triangulatie mogelijk maakt.

HSV-kleurdetectie voor objecten

Voor objectdetectie is gekozen voor HSV-drempelwaarden vanwege de eenvoud en lage rekenkosten. Het nadeel is dat de drempelwaarden zijn afgesteld op de echte wereld, terwijl de simulator net andere kleurwaarden gebruikt. Op een echte robot presteren de drempelwaarden beter.

Innovation gating in de EKF

Zonder gating zorgde de SLAM-correctie voor grote sprongen in de gefuseerde positie. De oorzaak ligt in het verschil tussen het cameraframe en het robotframe: de geëxtraheerde yaw uit de SLAM-pose correspondeert niet direct met de rijrichting van de robot. De gating zorgt dat alleen kleine, plausibele correcties worden doorgevoerd.

Monoculaire schaal

Monoculaire SLAM kan de absolute schaal niet bepalen. De kaart heeft de juiste vorm maar geen metrische afstanden. In een volledig systeem zou de odometrieschaal gebruikt worden om de SLAM-kaart op schaal te brengen.

4. Beperkingen en faalscenario's

Schaalambigüiteit

De SLAM-puntenwolk zweeft boven de odometrie-visualisatie in RViz. Dit is een direct gevolg van de monoculaire schaalambigüiteit gecombineerd met het verschil in referentiefraam tussen camera en robot. Zonder extrinsieke kalibratie van de camera ten opzichte van het robotframe kunnen de twee systemen niet in hetzelfde coördinatenstelsel worden geplaatst.

Simulator versus echte robot

In de Duckiematrix zijn de encoders vrijwel perfect: geen wielslip, geen ruis. Hierdoor is de odometrie in de simulator nauwkeuriger dan in de praktijk en is het voordeel van sensorfusie minder zichtbaar. Op een echte DB21 op een glibberige ondergrond zou de odometrie snel drift vertonen en zou de SLAM-correctie duidelijk verbeteren.

Weinig textuur

De ORB-detector heeft visuele structuur nodig om features te vinden. In omgevingen met weinig contrast (witte muren, lege vloer) worden te weinig features gevonden om de beweging te schatten. De node valt dan terug op puur de odometrie.

Snelle rotaties

Bij snelle draaiingen tussen twee opeenvolgende frames kunnen feature-matches mislukken. De Fundamental Matrix heeft dan niet genoeg inlier-correspondentie om betrouwbaar te zijn en de frame wordt overgeslagen. Dit resulteert in een korte onderbreking van de kaartopbouw.

Objectdetectie in de simulator

De HSV-drempelwaarden voor duckies en stoplichten zijn niet goed afgesteld op de simulatorkleuren. Hierdoor worden objecten in de Duckiematrix sporadisch of niet gedetecteerd. Op een echte robot met de juiste belichting werken de drempelwaarden betrouwbaarder.